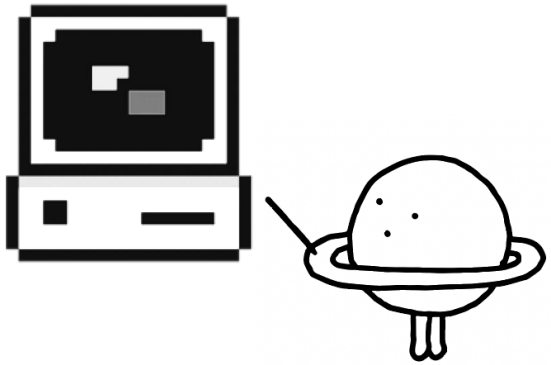


# Docker Engine, java TestContainer

## (feat. TestContainer 트러블슈팅)

우아한테크코스 BE 7기 메이

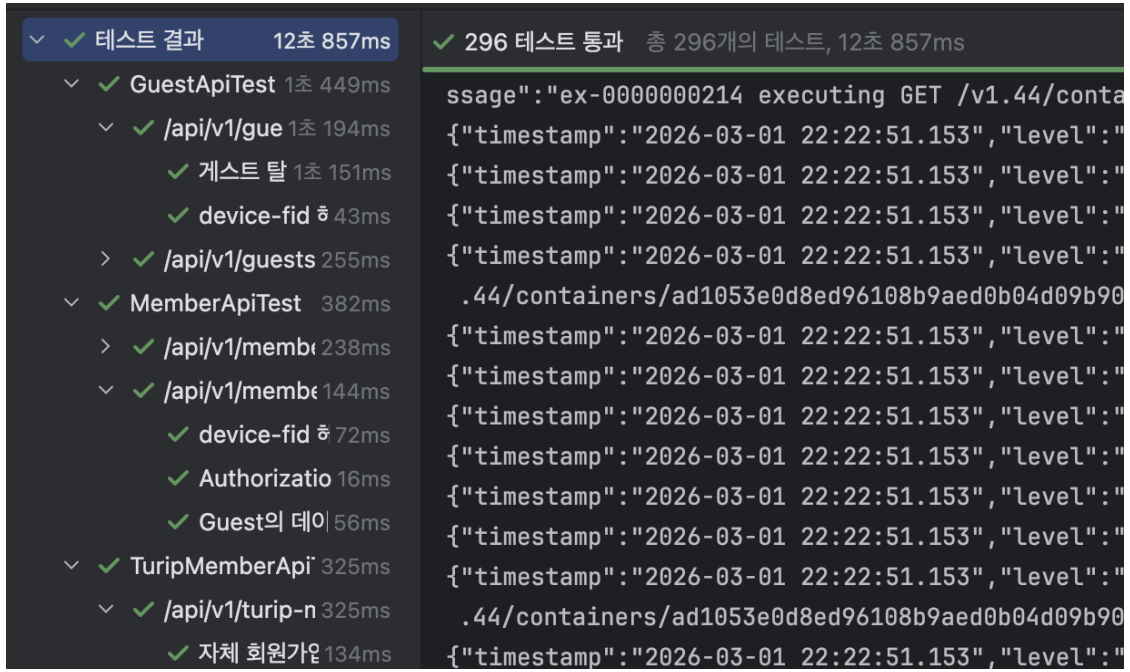
# 트러블슈팅

# 문제 상황

| | | |
|---|---|---|
| [feat] SSE 기반 공유 폴더 상태 스트림 API 구현<br>Dev App CI/CD #94: Pull request #581 synchronize by seaniiio | feature/#571-2 | Feb 13, 3:34 PM GMT+9<br>1m 16s |
| [feat] SSE 기반 공유 폴더 상태 스트림 API 구현<br>Dev App CI/CD #93: Pull request #581 synchronize by seaniiio | feature/#571-2 | Feb 13, 3:15 PM GMT+9<br>1m 16s |
| [feat] SSE 기반 공유 폴더 상태 스트림 API 구현<br>Dev App CI/CD #92: Pull request #581 synchronize by seaniiio | feature/#571-2 | Feb 13, 3:07 PM GMT+9<br>1m 17s |

**갑작스러운 CI 실패...**

# 문제 상황




```
AccountInsertRepositoryTest > insert에 성공한 경우 Account를 반환하고 DB에 저장된다 FAILED
    java.lang.IllegalStateException at DefaultCacheAwareContextLoaderDelegate.java:180
        Caused by: org.springframework.beans.factory.BeanDefinitionStoreException at ConfigurationClassEnhancer.java:136
            Caused by: org.springframework.cglib.core.CodeGenerationException at ConfigurationClassEnhancer.java:202
                Caused by: java.lang.IllegalStateException at LoadingCache.java:75
                    Caused by: java.lang.ExceptionInInitializerError at Class.java:-2
                        Caused by: java.lang.IllegalStateException at DockerClientProviderStrategy.java:274

AccountInsertRepositoryTest > nickname이 중복되면 Optional.empty()를 반환한다 FAILED
    java.lang.IllegalStateException at DefaultCacheAwareContextLoaderDelegate.java:145

AccountInsertRepositoryTest > 동시에 같은 nickname으로 insert 시도 시 하나만 성공한다 FAILED
    java.lang.IllegalStateException at DefaultCacheAwareContextLoaderDelegate.java:145

ContentKeywordApiTest > initializationError FAILED
    java.lang.IllegalStateException at DefaultCacheAwareContextLoaderDelegate.java:180
        Caused by: org.springframework.beans.factory.BeanDefinitionStoreException at ConfigurationClassEnhancer.java:136
            Caused by: org.springframework.cglib.core.CodeGenerationException at ConfigurationClassEnhancer.java:202
                Caused by: java.lang.IllegalStateException at LoadingCache.java:75
                    Caused by: java.lang.ExceptionInInitializerError at Class.java:-2
                        Caused by: java.lang.IllegalStateException at DockerClientProviderStrategy.java:274
```

**로컬은 성공하는데 왜 CI만?**

# 문제 상황

```java
@SpringBootTest
@ActiveProfiles("test")
@Import(TestContainerConfig.class)
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
class AccountInsertRepositoryTest {
```

```java
@DataJpaTest
@ActiveProfiles("test")
@Import(TestContainerConfig.class)
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
class ContentRepositoryTest {
```

```java
@ActiveProfiles("test")
@Import(TestContainerConfig.class)
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@Transactional(propagation = org.springframework.transaction.annotation.Propagation.NOT_SUPPORTED)
public class ContentKeywordApiTest {
```

# 문제 상황

```java
@SpringBootTest
@ActiveProfiles("test")
@Import(TestContainerConfig.class)
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
class AccountInsertRepositoryTest {
```

```java
@DataJpaTest
@ActiveProfiles("test")
@Import(TestContainerConfig.class)
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
class ContentRepositoryTest {
```

```java
@ActiveProfiles("test")
@Import(TestContainerConfig.class)
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@Transactional(propagation = org.springframework.transaction.annotation.Propagation.NOT_SUPPORTED)
public class ContentKeywordApiTest {
```

## MySQL 환경에서 테스트하기 위해 TestContainer를 사용하고 있었음...

```java
@TestConfiguration
public class TestContainerConfig {

    @Bean
    @ServiceConnection
    public MySQLContainer<?> mySQLContainer() {
        return new MySQLContainer<>( dockerImageName: "mysql:8.0.42")
                .withUsername("root")
                .withPassword("rootpwd")
                .withDatabaseName("test_db");
    }
}
```

# 해결 시도 1 - AI

오 이 스택트레이스 마지막 줄 `DockerClientProviderStrategy.java:229` 이거면 **거의 99% "Testcontainers가 도커를 못 찾음 / 못 붙음"** 이 케이스야. 그래서 스프링 컨텍스트 뜨는 도중에(= initializationError) DB 같은 컨테이너 빈 만들다가 터진 거고, 그 여파로 `UnsatisfiedDependencyException` 줄줄이 따라온 거.

아래 순서대로 보면 빨리 잡혀.

---

### 1) 제일 흔한 원인: 도커가 안 켜져 있음

켜져있음

- 로컬이면 **Docker Desktop 실행**되어 있어야 해.
- 터미널에서 바로 확인:

```bash
docker ps
```

여기서 에러 나면(daemon not running / cannot connect) → 테스트는 무조건 터짐.

---

### 2) CI/서버/리눅스에서 도커 접근 권한 문제

권한있음

리눅스에서 자주 나오는 케이스:

- `permission denied while trying to connect to the Docker daemon socket`

해결:

# 해결 시도 2 – 레포 찾아가기

Closed **[Bug]: Incompatibility in Ubuntu Github runners with Docker Engine 29.1.*** #11491

conductor-oss/... #772

## Docker version

```
29.1.*
```

## What happened? Github ubuntu runner에서 사용하는 docker 버전이 올라감

Github's Ubuntu runners got upgraded to Docker 29.1. (https://github.com/actions/runner-images/blob/main/images/ubuntu/Ubuntu2404-Readme.md)

As a side effect, builds using testcontainers-java, even on the latest version, start failing.

I see the latest scheduled runs on your CI, possibly spot the issue: https://github.com/testcontainers/testcontainers-java/actions/runs/21969291388

## Relevant log output

```
java.lang.IllegalStateException at DefaultCacheAwareContextLoaderDelegate.java:180
        Caused by: java.lang.IllegalStateException at SpringApplication.java:823
            Caused by: java.lang.ExceptionInInitializerError at Unsafe.java:1160
                Caused by: java.lang.IllegalStateException at DockerClientProviderStrategy.java:274
```

## Additional Information

# 해결 시도 2 – 레포 찾아가기

actions / runner-images

[Windows/Ubuntu] Docker Server and Client will be updated to version 29.1.*, Docker Compose will be updated to version 2.40.3 on February 9th, 2026 #13474

Open

**2026년 2월 9일부터 ubuntu 24.04 이미지에서 사용하는 Docker Engine 버전이 올라감**

**Possible impact**

Workflows that rely on deprecated or changed Docker behavior may require adjustments after the upgrade.

**Platforms affected**

- [x] Azure DevOps
- [x] GitHub Actions

**Runner images affected**

- [x] Ubuntu 22.04
- [x] Ubuntu 24.04
- [ ] Ubuntu Slim
- [ ] macOS 14
- [ ] macOS 14 Arm64
- [ ] macOS 15
- [ ] macOS 15 Arm64
- [ ] macOS 26 Arm64
- [ ] Windows Server 2019
- [x] Windows Server 2022
- [x] Windows Server 2025

# 문제 원인 정리

**테스트에서 사용하는 Docker API 버전**

```
if (configBuilder.build().getApiVersion() == RemoteApiVersion.UNKNOWN_VERSION) {
    configBuilder.withApiVersion(RemoteApiVersion.VERSION_1_32);
}
```

**호환되지 않음**

Breaking Changes

**Docker Engine 29.1.****

- The Go module `github.com/docker/docker` is deprecated in favor of `github.com/moby/moby/client` and `github.com/moby/moby/api`. The `github.com/moby/moby` module is considered an **internal implementation detail** - the only supported public modules are `client` and `api`. Starting with v29, releases are tagged with the `docker-` prefix (e.g., `docker-v29.0.0`). **This only affects Go module users and package maintainers.**
- The daemon now requires API version `v1.44` or later (Docker v25.0+).

# 테스트 로컬 환경

```
❯ docker version
Client: Docker Engine - Community
 Version:           28.0.1
 API version:       1.47 (downgraded from 1.48)
 Go version:        go1.24.0
 Git commit:        068a01ea94
 Built:             Tue Feb 25 17:52:55 2025
 OS/Arch:           darwin/arm64
 Context:           desktop-linux

Server: Docker Desktop 4.38.0 (181591)
 Engine:
  Version:          27.5.1
  API version:      1.47 (minimum version 1.24)
  Go version:       go1.22.11
```

# CI와 같은 환경

```
❯ docker version
Client: Docker Engine - Community
 Version:           28.0.1
 API version:       1.48
 Go version:        go1.24.0
 Git commit:        068a01ea94
 Built:             Tue Feb 25 17:52:55 2025
 OS/Arch:           darwin/arm64
 Context:           desktop-linux

Server: Docker Desktop 4.63.0 (220185)
 Engine:
  Version:          29.2.1
  API version:      1.53 (minimum version 1.44)
  Go version:       go1.25.6
```

## 테스트 성공

✓ 285 테스트 통과 총 285개의 테스트, 14초 17ms

mysql 1/634

```
MINUTES"}
{"timestamp":"2026-03-04 14:56:00.267","level":"DEBUG","message":"http-outgoing-2 << \"18d0[\\r][\\n]\""}
{"timestamp":"2026-03-04 14:56:00.267","level":"DEBUG","message":"http-outgoing-2 << "
"{\"Id\":\"200e4438e0afba555470117d0ccd13a5221bf166f13e59c27fbad795d13145db\",\"Created\":\"2026-03-04T05:55:57
.059020174Z\",\"Path\":\"docker-entrypoint.sh\",\"Args\":[\"mysqld\"],\"State\":{\"Status\":\"running\",
\"Running\":true,\"Paused\":false,\"Restarting\":false,\"OOMKilled\":false,\"Dead\":false,\"Pid\":10561,\"ExitCode\":0,
\"Error\":\"\",\"StartedAt\":\"2026-03-04T05:55:57.184842591Z\",\"FinishedAt\":\"0001-01-01T00:00:00Z\"},
```

## 테스트 실패

✗ 5개의 테스트가 실패하고, 262개가 통과했습니다 총 267개의 테스트, 6초 767ms

```
.system/bootstrap/usr/bin, /var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/appleinternal/bin,
/Library/Apple/usr/bin, /Users/siwon/Library/Application Support/JetBrains/Toolbox/scripts, /Users/siwon/go/bin])"}
{"timestamp":"2026-03-04 15:14:14.548","level":"ERROR","message":"Could not find a valid Docker environment. Please check
configuration. Attempted configurations were:\n\tUnixSocketClientProviderStrategy: failed with exception BadRequestException
(Status 400: {\"ID\":\"\",\"Containers\":0,\"ContainersRunning\":0,\"ContainersPaused\":0,\"ContainersStopped\":0,\"Images\":0,
\"Driver\":\"\",\"DriverStatus\":null,\"Plugins\":{\"Volume\":null,\"Network\":null,\"Authorization\":null,\"Log\":null},
\"MemoryLimit\":false,\"SwapLimit\":false,\"CpuCfsPeriod\":false,\"CpuCfsQuota\":false,\"CPUShares\":false,\"CPUSet\":false,
\"PidsLimit\":false,\"IPv4Forwarding\":false,\"Debug\":false,\"NFd\":0,\"OomKillDisable\":false,\"NGoroutines\":0,
\"SystemTime\":\"\",\"LoggingDriver\":\"\",\"CgroupDriver\":\"\",\"NEventsListener\":0,\"KernelVersion\":\"\",
\"OperatingSystem\":\"\",\"OSVersion\":\"\",\"OSType\":\"\",\"Architecture\":\"\",\"IndexServerAddress\":\"\",
\"RegistryConfig\":null,\"NCPU\":0,\"MemTotal\":0,\"GenericResources\":null,\"DockerRootDir\":\"\",\"HttpProxy\":\"\",
\"HttpsProxy\":\"\",\"NoProxy\":\"\",\"Name\":\"\",\"Labels\":[\"com.docker.desktop
.address=unix:///Users/siwon/Library/Containers/com.docker.docker/Data/docker-cli.sock\"],\"ExperimentalBuild\":false,
\"ServerVersion\":\"\",\"Runtimes\":null,\"DefaultRuntime\":\"\",\"Swarm\":{\"NodeID\":\"\",\"NodeAddr\":\"\",
```

# 문제 해결

## Set default docker API version to 1.44 #11216

Merged eddumelendez merged 3 commits into `main` from `switch-docker-api-144` on Nov 13, 2025

Milestone

2.0.2

Conversation 3 | Commits 3 | Checks 99 | Files changed 3

**eddumelendez** commented on Nov 13, 2025 Member

Fixes #11212

1 3 9

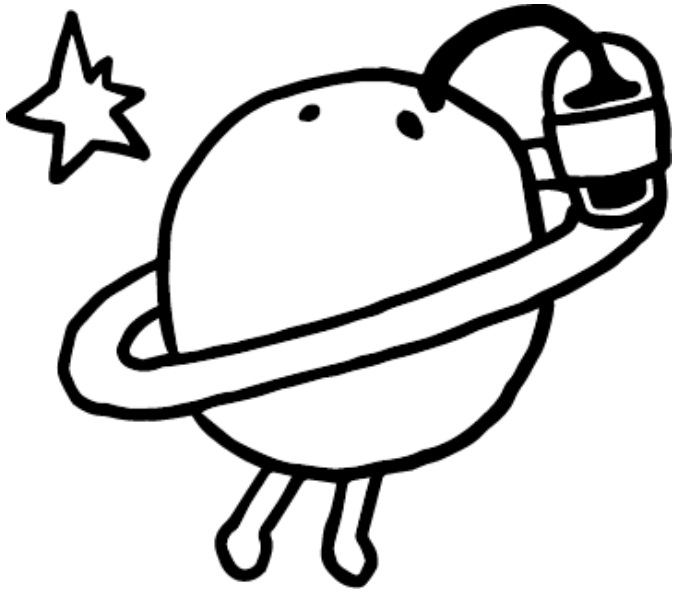
**TestContainer 2.0.2버전부터 docker api 버전 업그레이드**

```
testImplementation "org.testcontainers:testcontainers-junit-jupiter:2.0.2"
testImplementation "org.testcontainers:testcontainers:2.0.2"
testImplementation "org.testcontainers:testcontainers-mysql:2.0.2"
```

# 궁금증

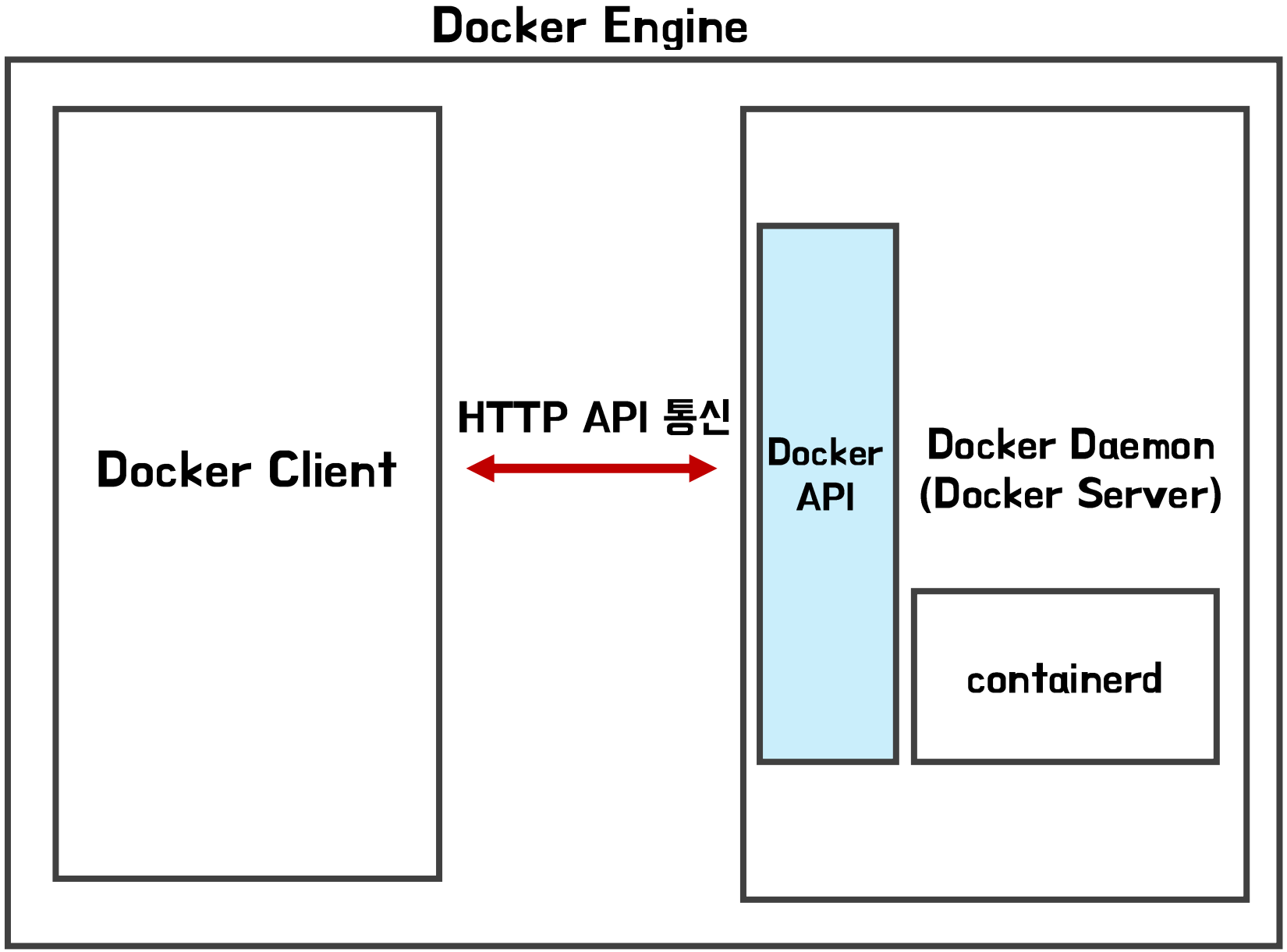
TestContainer의 동작 방식?
Docker Engine 구조?

# Docker Engine, Docker API



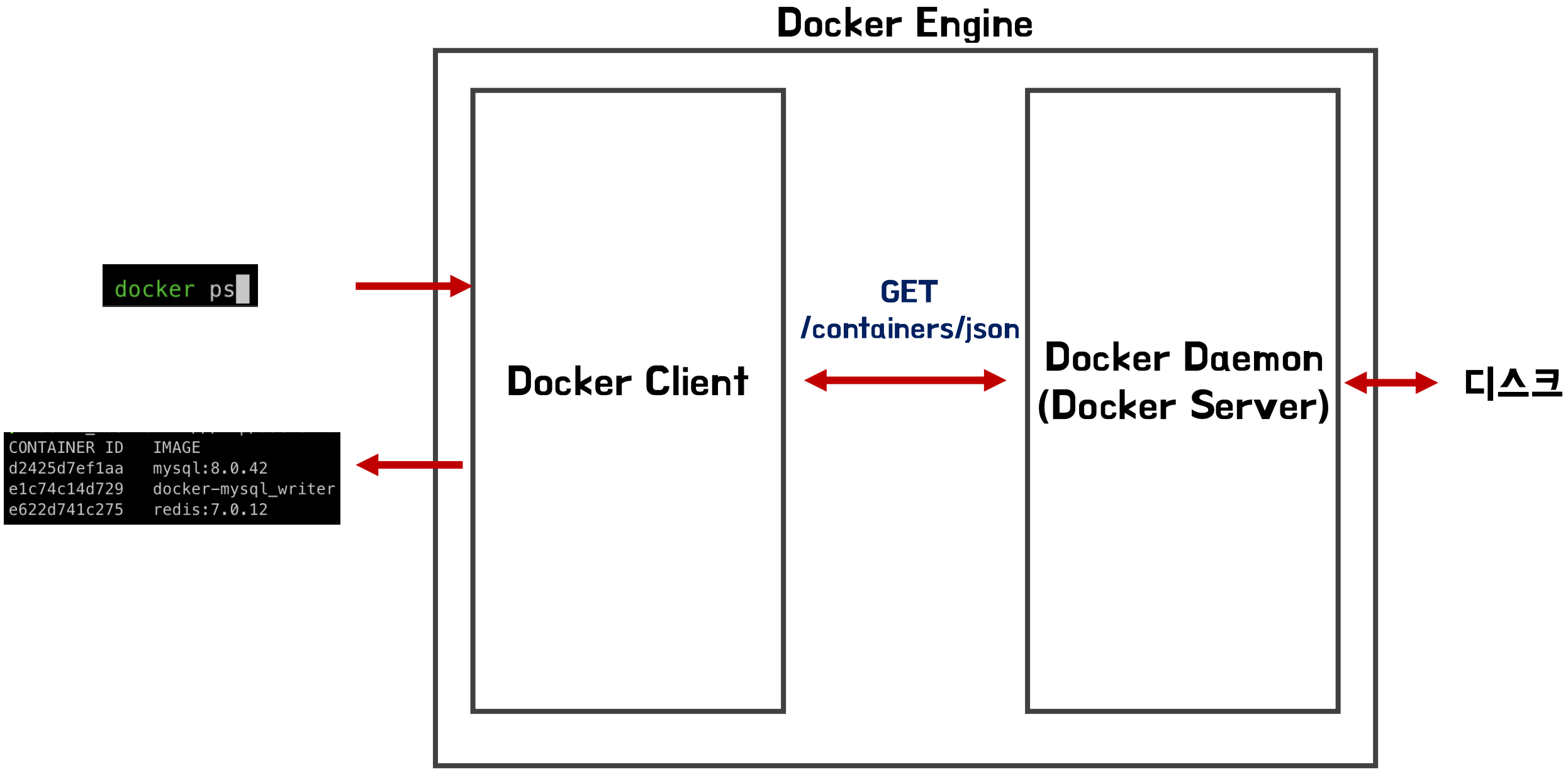
# Docker Client, Server

사용자의 명령을
API로 변환해서
Daemon에 전달

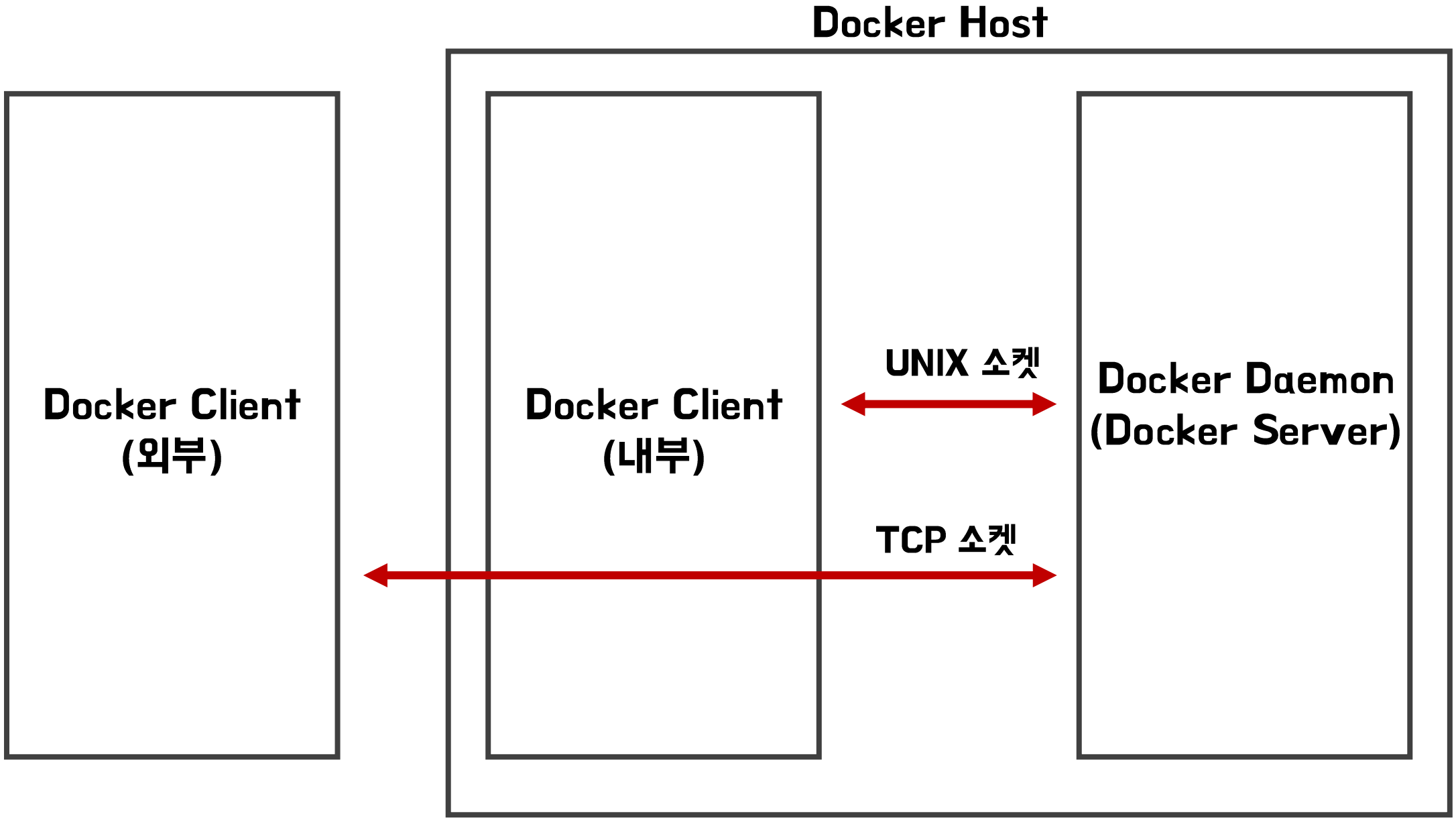



API 서빙
컨테이너 관리
이미지 빌드
빌드 캐시
...

# Docker Client, Server



# Docker Client, Server



# Docker API

docker docs

Search...

Containers

- GET List containers
- POST Create a container
- GET Inspect a container
- GET List processes running inside a container
- GET Get container logs
- GET Get changes on a container's filesystem
- GET Export a container
- GET Get container stats based on resource usage
- POST Resize a container TTY
- POST Start a container
- POST Stop a container

GET /containers/json

## Response samples

200 400 500

Content type

application/json

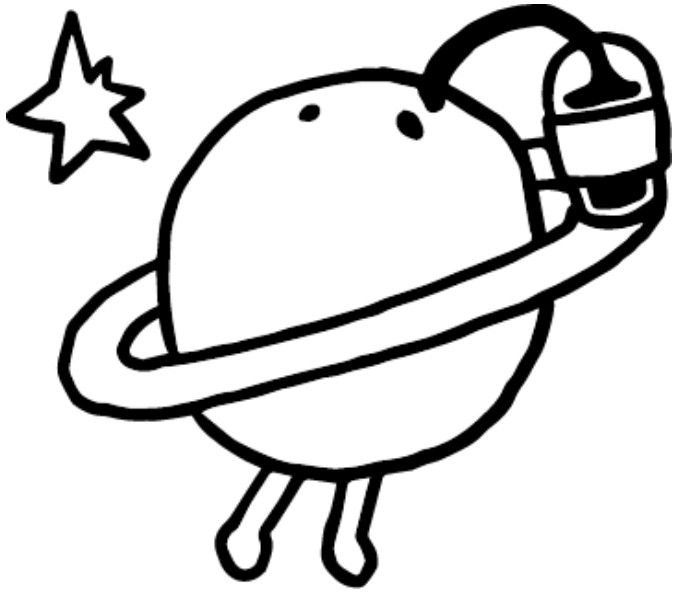
Copy Expand all Collapse all


```
[
 - {
      "Id": "8dfafdbc3a40",
    + "Names": [ … ],
      "Image": "ubuntu:latest",
      "ImageID": "d74508fb6632491cea586a1fd7d748dfc5274cd6fdfedee309ecdcbc2
      "Command": "echo 1",
      "Created": 1367854155,
      "State": "Exited",
```

# Docker API

```
❯ curl --unix-socket /var/run/docker.sock http://localhost/containers/json | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  5565    0  5565    0     0   1848      0 --:--:--  0:00:03 --:--:--  1848
[
  {
    "Id": "d2425d7ef1aaeccf9cf68ee9ffcc3300fe8fdcaf6f5f7a08fedb2d6c4c2e0aae",
    "Names": [
      "/turip-mysql-dev"
    ],
    "Image": "mysql:8.0.42",
    "ImageID": "sha256:9440831d622bc47a5ba1b810a4305c74b99d67d2374cc4d69fa7dfa602610782",
    "Command": "docker-entrypoint.sh --ngram_token_size=1",
    "Created": 1763447489,
    "Ports": [
      {
        "PrivatePort": 33060,
        "Type": "tcp"
      },
      {
        "IP": "0.0.0.0",
        "PrivatePort": 3306,
        "PublicPort": 3306,
        "Type": "tcp"
      }
    ],
    "Labels": {
```

# Java TestContainer



# DockerClient

```
private void tryStart() {
    try {
        String dockerImageName = getDockerImageName();
        logger().debug("Starting container: {}", dockerImageName);

        Instant startedAt = Instant.now();
        logger().info("Creating container for image: {}", dockerImageName);
        CreateContainerCmd createCommand = dockerClient.createContainerCmd(dockerImageName);
        applyConfiguration(createCommand);
```

TestContainer에서 컨테이너 생성할 때

```
public class CreateContainerCmdExec extends AbstrSyncDockerCmdExec<CreateContainerCmd, CreateConta

    protected CreateContainerResponse execute(CreateContainerCmd command) {
        WebTarget webResource = this.getBaseResource().path(new String[]{"/containers/create"});
```

컨테이너 생성 docker api

# Java TestContainer 동작 분석 방법

로그를 보면서
Test container의
동작 흐름을 살펴보자

## <테스트 환경>

```java
@TestConfiguration
public class TestContainerConfig {

    @Bean
    @ServiceConnection
    public MySQLContainer mySQLContainer() {
        return new MySQLContainer( dockerImageName: "mysql:8.0.42")
                .withUsername("root")
                .withPassword("rootpwd")
                .withDatabaseName("test_db");
    }
}
```

```java
package org.testcontainers.mysql;

import ...

  Testcontainers implementation for MySQL.
  Supported image: mysql
  Exposed ports: 3306

public class MySQLContainer extends JdbcDatabaseContainer<MySQLContainer> {
```

# 로그를 통해 테스트 실행 순서 파악하기

## 1. 도커 연결 시도(unix 소켓)

```
{"timestamp":"2026-03-05 18:58:06.062","level":"DEBUG","message":"Checking Docker OS type for local Unix socket (unix:///var/run/docker.sock)"}
{"timestamp":"2026-03-05 18:58:06.063","level":"INFO","message":"Docker host IP address is localhost"}
```

## 2. 도커 엔진에 대한 정보 확인 & 연결

```
{"timestamp":"2026-03-05 18:58:06.081","level":"DEBUG","message":"Docker version: {Components=[{Name=Engine, Version=29.2.1, Details={ApiVersion=1.53, Arch=arm64,
 BuildTime=2026-02-02T17:16:47.000000000+00:00, Experimental=false, GitCommit=6bc6209, GoVersion=go1.25.6, KernelVersion=6.12.72-linuxkit, MinAPIVersion=1.44, Os=linux}},
 {Name=containerd, Version=v2.2.1, Details={GitCommit=dea7da592f5d1d2b7755e3a161be07f43fad8f75}}, {Name=runc, Version=1.3.4, Details={GitCommit=v1.3.4-0-gd6d73eb8}},
 {Name=docker-init, Version=0.19.0, Details={GitCommit=de40ad0}}], KernelVersion=6.12.72-linuxkit, ApiVersion=1.53, Platform={Name=Docker Desktop 4.63.0 (220185)}, Os=linux,
 GitCommit=6bc6209, Version=29.2.1, MinAPIVersion=1.44, Arch=arm64, GoVersion=go1.25.6, BuildTime=2026-02-02T17:16:47.000000000+00:00}"}
```

```
{"timestamp":"2026-03-05 18:58:06.081","level":"INFO","message":"Connected to docker: \n  Server Version: 29.2.1\n  API Version: 1.53\n  Operating System: Docker Desktop\n
 Total Memory: 7836 MB\n  Labels: \n    com.docker.desktop.address=unix:///Users/siwon/Library/Containers/com.docker.docker/Data/docker-cli.sock"}
```

## 3. 이미지 pull & 컨테이너 생성

```
{"timestamp":"2026-03-05 18:58:06.534","level":"DEBUG","message":"Using locally available and not pulling image: mysql:8.0.42"}
{"timestamp":"2026-03-05 18:58:06.535","level":"INFO","message":"Creating container for image: mysql:8.0.42"}
```

# 로그를 통해 테스트 실행 순서 파악하기

## 4. TestContainer 연결 시도

```
{"timestamp":"2026-03-05 18:58:06.699","level":"INFO","message":"Waiting for database connection to become available at jdbc:mysql://localhost:55614/test_db using query
 'SELECT 1'"}
{"timestamp":"2026-03-05 18:58:06.699","level":"DEBUG","message":"Cmd: 96cdb3c9139c0b05730f217886473318f0ff2baab672f66b8b0c7ccef74c2068,false"}
{"timestamp":"2026-03-05 18:58:06.699","level":"DEBUG","message":"GET:
 DefaultWebTarget{path=[/containers/96cdb3c9139c0b05730f217886473318f0ff2baab672f66b8b0c7ccef74c2068/json], queryParams={}}"}
{"timestamp":"2026-03-05 18:58:06.711","level":"DEBUG","message":"Cmd: 96cdb3c9139c0b05730f217886473318f0ff2baab672f66b8b0c7ccef74c2068,false"}
{"timestamp":"2026-03-05 18:58:06.711","level":"DEBUG","message":"GET:
 DefaultWebTarget{path=[/containers/96cdb3c9139c0b05730f217886473318f0ff2baab672f66b8b0c7ccef74c2068/json], queryParams={}}"}
{"timestamp":"2026-03-05 18:58:06.890","level":"DEBUG","message":"Cmd: 96cdb3c9139c0b05730f217886473318f0ff2baab672f66b8b0c7ccef74c2068,false"}
{"timestamp":"2026-03-05 18:58:06.890","level":"DEBUG","message":"GET:
```

## 5. 테스트 실행

```
        WHERE
            MATCH(p.name) AGAINST(? IN BOOLEAN MODE)
```

## 6. 컨테이너 정리

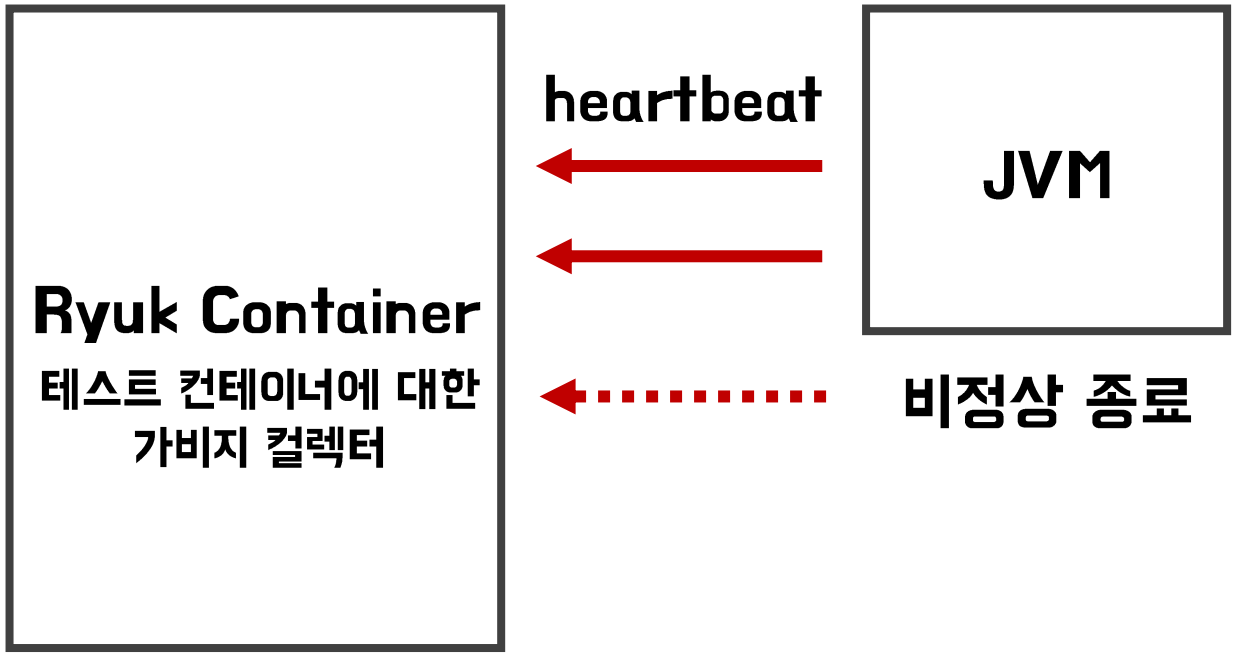
```
{"timestamp":"2026-03-05 18:58:19.919","level":"DEBUG","message":"Removed container and associated volume(s): mysql:8.0.42"}
```

# TestContainer 확인하기

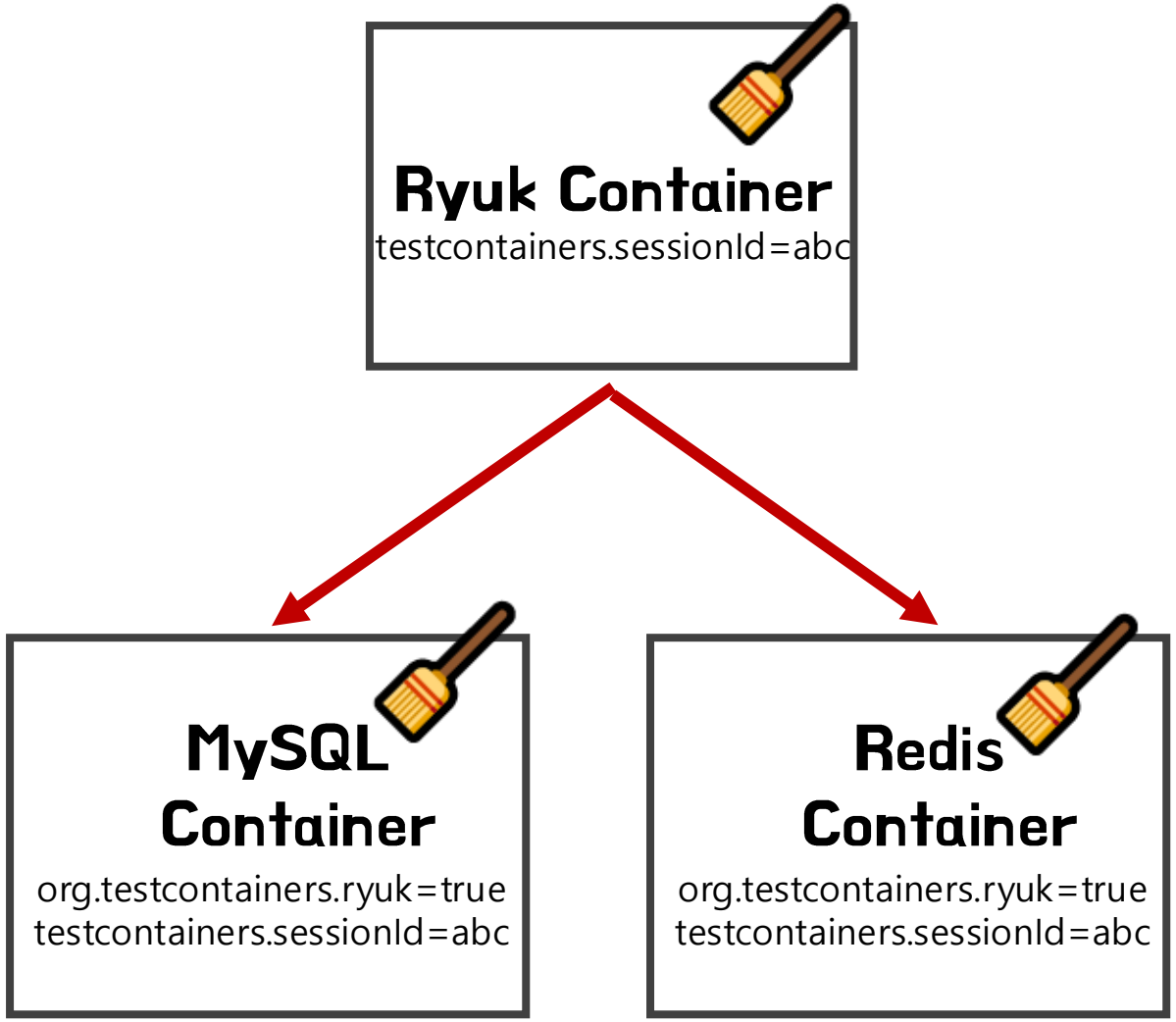
```
❯ docker ps
CONTAINER ID   IMAGE                  COMMAND                   CR
d2425d7ef1aa   mysql:8.0.42           "docker-entrypoint.s…"    3 m
e1c74c14d729   docker-mysql_writer    "docker-entrypoint.s…"    4 m
e622d741c275   redis:7.0.12           "docker-entrypoint.s…"    4 m
❯ docker ps
CONTAINER ID   IMAGE                          COMMAND
3848b94087a2   mysql:8.0.42                   "docker-entrypoint.s…
3e65b0d4b1c7   testcontainers/ryuk:0.13.0     "/bin/ryuk"
k-de799d17-fd64-4ec2-9815-f83acb5da0b7
d2425d7ef1aa   mysql:8.0.42                   "docker-entrypoint.s…
e1c74c14d729   docker-mysql_writer            "docker-entrypoint.s…
e622d741c275   redis:7.0.12                   "docker-entrypoint.s…
```

```
"message":"Ryuk started - will monitor and terminate Testcontainers containers on JVM exit"}
```

# Ryuk Container



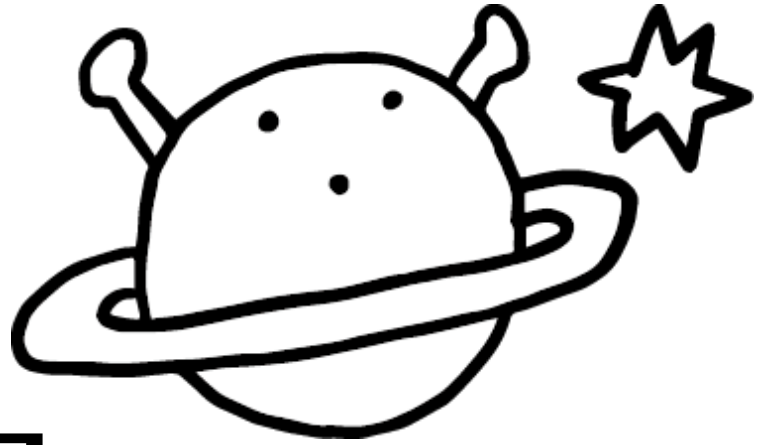
# Ryuk Container



# Ryuk Container

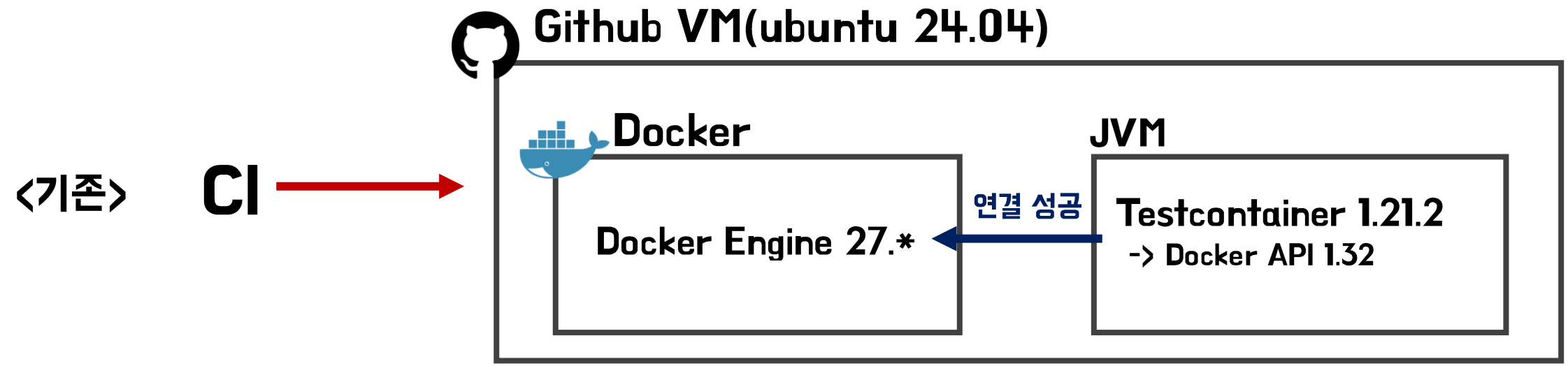
Testcontainers

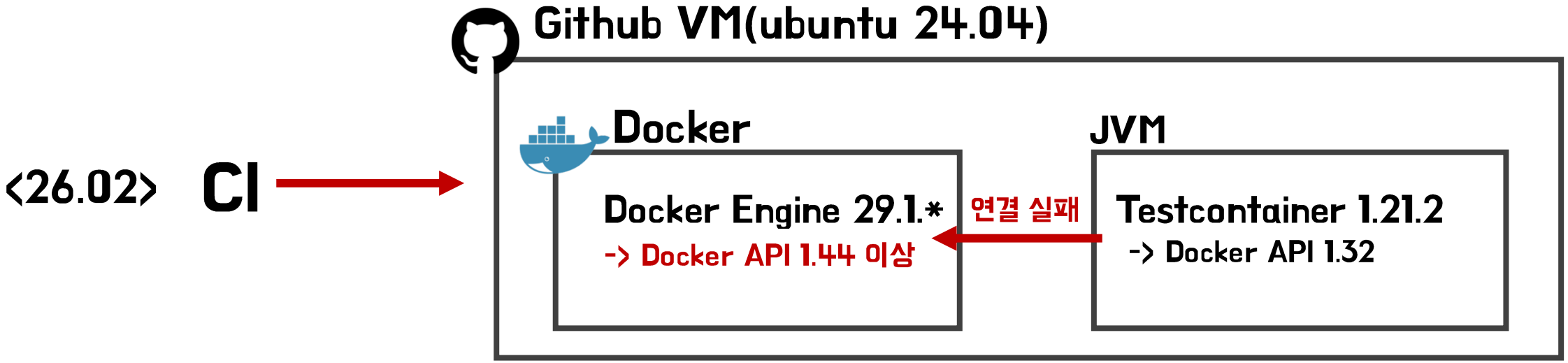
Moby Ryuk derives its name from the anime character Ryuk and is a fitting choice due to the intriguing narrative of the anime Death Note.



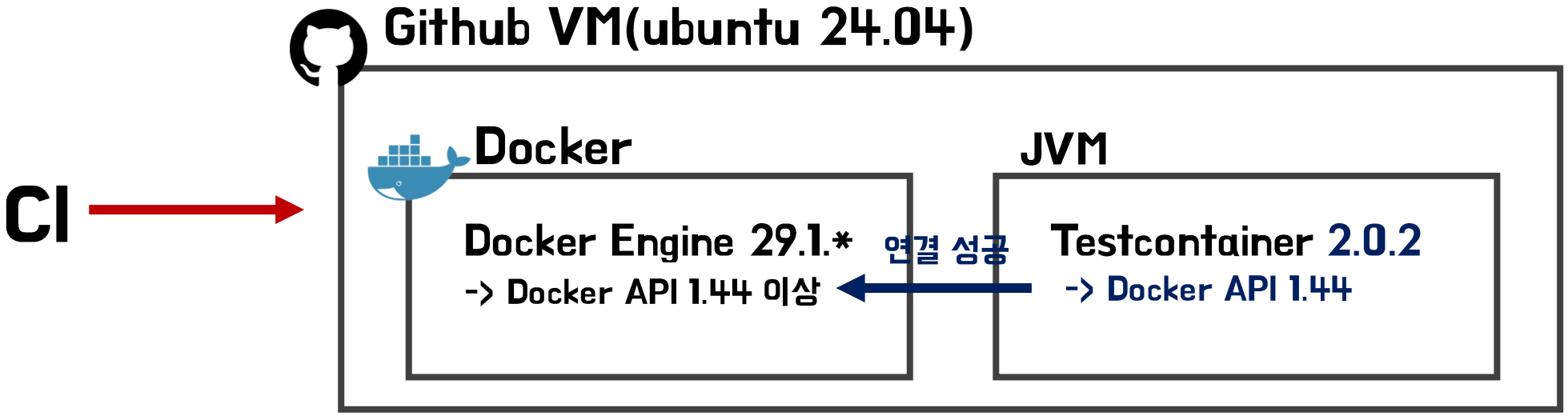
# 결론

# 이슈 상황 정리

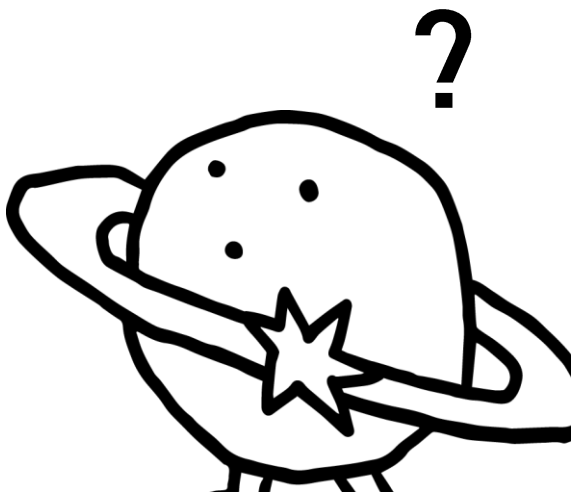

<기존>
CI
Github VM(ubuntu 24.04)
Docker
Docker Engine 27.*
연결 성공
JVM
Testcontainer 1.21.2
-> Docker API 1.32


<26.02>
CI
Github VM(ubuntu 24.04)
Docker
Docker Engine 29.1.*
-> Docker API 1.44 이상
연결 실패
JVM
Testcontainer 1.21.2
-> Docker API 1.32

# 이슈 상황 정리


Github VM(ubuntu 24.04)
CI
Docker
Docker Engine 29.1.*
-> Docker API 1.44 이상
연결 성공
JVM
Testcontainer 2.0.2
-> Docker API 1.44

# 느낀점

- 어떤 라이브러리를 사용하다가 문제가 생겼을 때는 이슈에서 찾아보자.

# Q&A